

TASK: USER PROFILE MANAGEMENT SYSTEM

Goal:

Students must understand **when to use which HTTP method** and **why**, by building a **mini real-world system**.

SCENARIO

You are working in a company building a **User Profile Dashboard**.

The backend already has **user data** stored.

Your frontend must allow:

- 1** View profile
- 2** Create profile
- 3** Update full profile
- 4** Update only one field
- 5** Delete profile

 Each action must use the **correct HTTP method**

DATA MODEL

User object:

```
{  
  id: 1,  
  name: "Manish",  
  age: 25,  
  role: "Developer"  
}
```

TASK BREAKDOWN

◆ TASK 1: VIEW USER PROFILE (GET)

 Real-life meaning

User opens profile page → sees details

 **Method to use**

 **GET**

 **Task instruction**

- Create a **GET API**
- Fetch user data
- Display it on UI

◆ **TASK 2: CREATE USER PROFILE (POST)**

 **Real-life meaning**

New user signs up

 **Method to use**

 **POST**

 **Task instruction**

- Create a form (name, age, role)
- Send data to backend
- Backend stores user
- Show success message

◆ **TASK 2: CREATE USER PROFILE (POST)**

 **Real-life meaning**

New user signs up

 **Method to use**

 **POST**

 **Task instruction**

- Create a form (name, age, role)
- Send data to backend
- Backend stores user

- Show success message

TASK 4: UPDATE ONLY ONE FIELD (PATCH)

Real-life meaning

User changes only **role** or **age**

Method to use

PATCH

Task instruction

- Update only one field
- Other data must remain untouched
- Use spread operator correctly

◆ TASK 5: DELETE USER PROFILE (DELETE)

Real-life meaning

User deletes account permanently

Method to use

DELETE

Task instruction

- Remove user from backend
- UI should update instantly
- Handle “already deleted” case